

Lab 2 — Cloud Data, Stat 214, Spring 2025

1 Introduction

Detecting clouds over polar regions from satellite imagery is difficult because ice and snow look nearly identical to clouds in single-angle imagery.

NASA’s Multi-angle Imaging SpectroRadiometer (MISR) [7] addresses this by imaging each scene from nine viewing angles. Clouds shift position across angles due to parallax, producing three expert features: NDAI (normalized angular displacement), SD (multi-angle radiance variability), and CORR (angular correlation) [1].

We have 164 MISR images, three with expert pixel-level cloud labels. Our goal is a binary classifier that generalizes to new polar scenes. We proceed in four steps: exploratory analysis (Section 2), feature engineering (Section 3), model training and evaluation (Section 4), and deployment with stability analysis (Section 4.6).

2 EDA

2.1 Dataset Overview

The dataset contains 164 MISR images over Arctic polar regions. Each image is an $N \times K$ matrix: N pixels (varying by image), two coordinate columns (y, x), eight feature columns (NDAI, SD, CORR, and five radiance channels DF, CF, BF, AF, AN), and for the three labeled images, an expert label column (+1 = cloud, -1 = non-cloud, 0 = unlabeled).

Table 1 summarizes the three labeled images. They differ in size and class balance: cloud fraction ranges from 28.9% to 47.8%, with an overall rate of about 39%. These differences create a natural test of cross-image generalization.

Table 1: Dataset overview: three labeled MISR images used for 3-fold cross-validation.

Image	Pixels	Cloud Fraction	Role (rotates across folds)
O012791	54,772	38.8%	Train / Val / Test
O013257	70,826	28.9%	Train / Val / Test
O013490	82,083	47.8%	Train / Val / Test

2.2 Expert Label Maps

Figure 1 shows the spatial layout of expert labels for a held-out test image. Cloud (red) and non-cloud (blue) pixels form large contiguous patches rather than scattered points, reflecting strong spatial autocorrelation. This has a direct consequence for evaluation: randomly splitting pixels into train and test sets would leak spatial information, inflating performance estimates.



Figure 1: Ground truth and model predictions for a held-out test image (Fold 1). The leftmost panel shows expert labels; subsequent panels show each model’s predictions. Cloud (red) and non-cloud (blue) regions form coherent spatial clusters.

2.3 Feature Distributions by Class

NDAI provides the strongest univariate separation between cloud and non-cloud pixels. SD is right-skewed with clouds showing a longer tail, motivating a log transform. CORR shows moderate separation, with non-cloud pixels having higher angular correlation. The five radiance channels overlap substantially but have consistent mean shifts, so their differences and ratios may carry useful information.

2.4 Feature Correlations

Figure 2 shows the Pearson correlation matrix. The five radiance channels are highly correlated ($r > 0.8$), as expected since they measure the same scene from different angles. NDAI, SD, and CORR are only weakly correlated with the radiance channels, confirming that expert features and raw radiances provide complementary information. This motivates including both families and constructing cross-feature interactions.

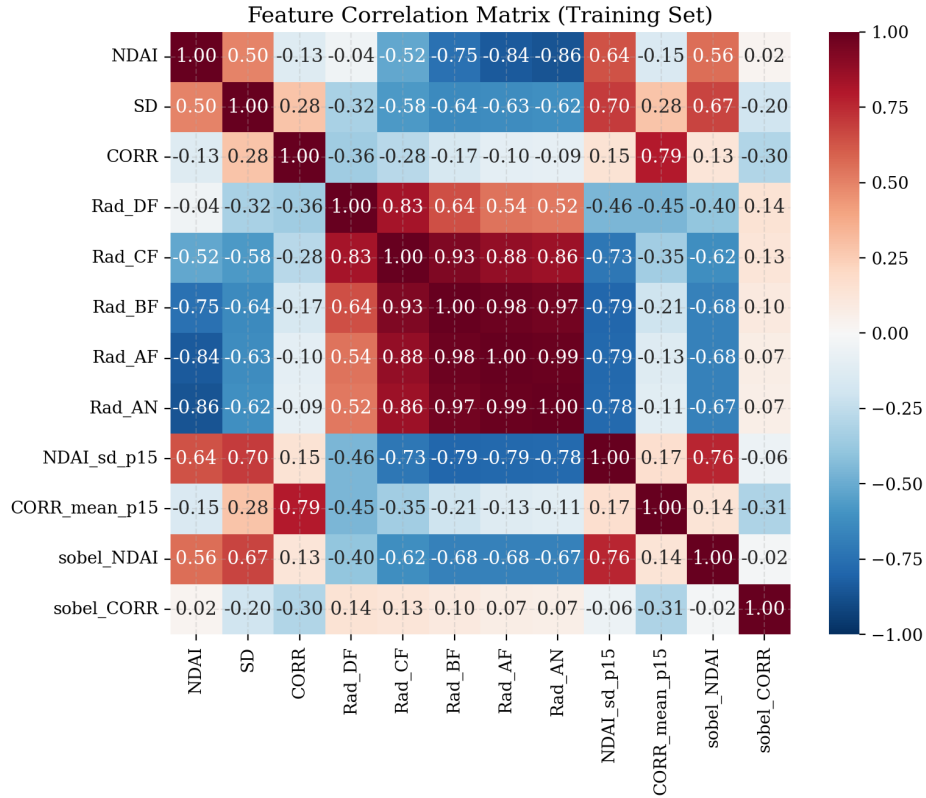


Figure 2: Pearson correlation matrix of the 8 raw features. The five radiance channels form a tightly correlated block; expert features (NDAI, SD, CORR) add complementary information.

2.5 Data Splitting Strategy

With only three labeled images, the splitting strategy matters a lot. Random pixel-level splits would cause severe data leakage because neighboring pixels carry nearly identical information. We confirm this quantitatively. Global Moran’s I exceeds 0.75 for every feature and image, and approaches 1.0 for the label itself, indicating strong spatial clustering. This rules out random splits and motivates **3-fold image-level cross-validation**: in each fold, one image is used for training (with an internal 80/20 stratified split), one for validation (threshold tuning only), and one as the held-out test set.

Table 2: Three-fold image-level cross-validation design.

Fold	Train (80% + 20% val)	Train images	Test
1	O013257 + O013490	122,328 pixels	O012791
2	O013490 + O012791	109,484 pixels	O013257
3	O012791 + O013257	100,479 pixels	O013490

Within each fold, 80% of the training pixels are used for model fitting and 20% are held out for threshold selection (finding the F1-maximizing decision threshold). No model parameters are updated using validation data, and all hyperparameters are fixed a priori. The test image is never seen during training or threshold selection. Because each split uses entirely different images, there is no spatial leakage. All random operations (train/val splits, model initialization, bootstrap resampling) use a fixed seed (42) for reproducibility. The cross-fold variance in our results reflects the genuine difficulty of generalizing across polar scenes with different cloud morphology.

The class imbalance (cloud fraction varies from 28.9% to 47.8% across images) means raw accuracy is misleading. We use class-weighted loss functions where applicable, report balanced accuracy alongside F1, and optimize thresholds via the precision-recall curve.

2.6 Distribution Shift Across Images

Figure 3 compares the feature distributions of the three labeled images. While NDAI and SD have broadly similar shapes, CORR and Rad_AN show noticeable shifts between images. These distributional differences help explain the cross-fold performance variation in Section 4.

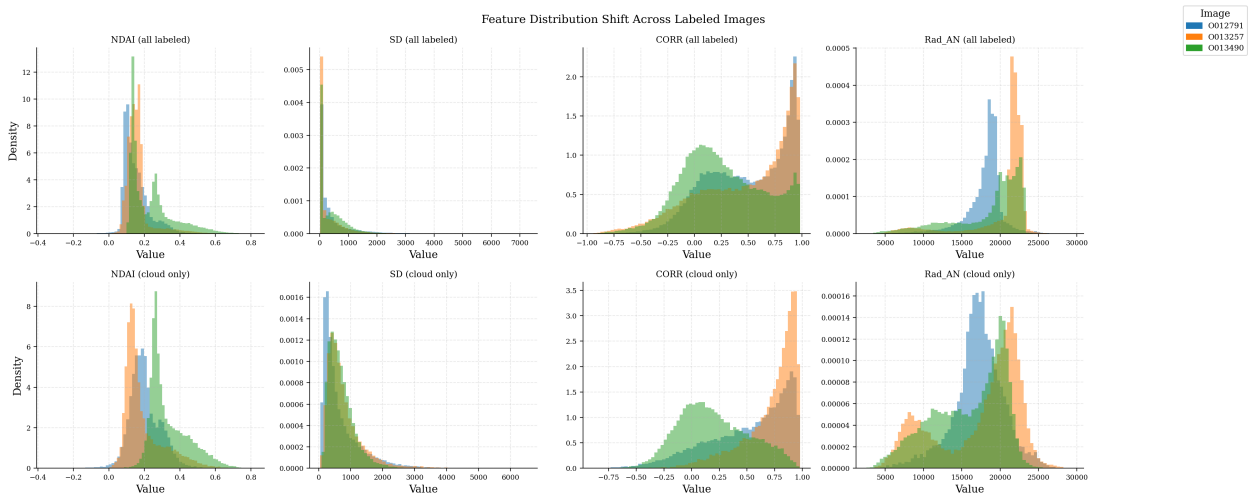
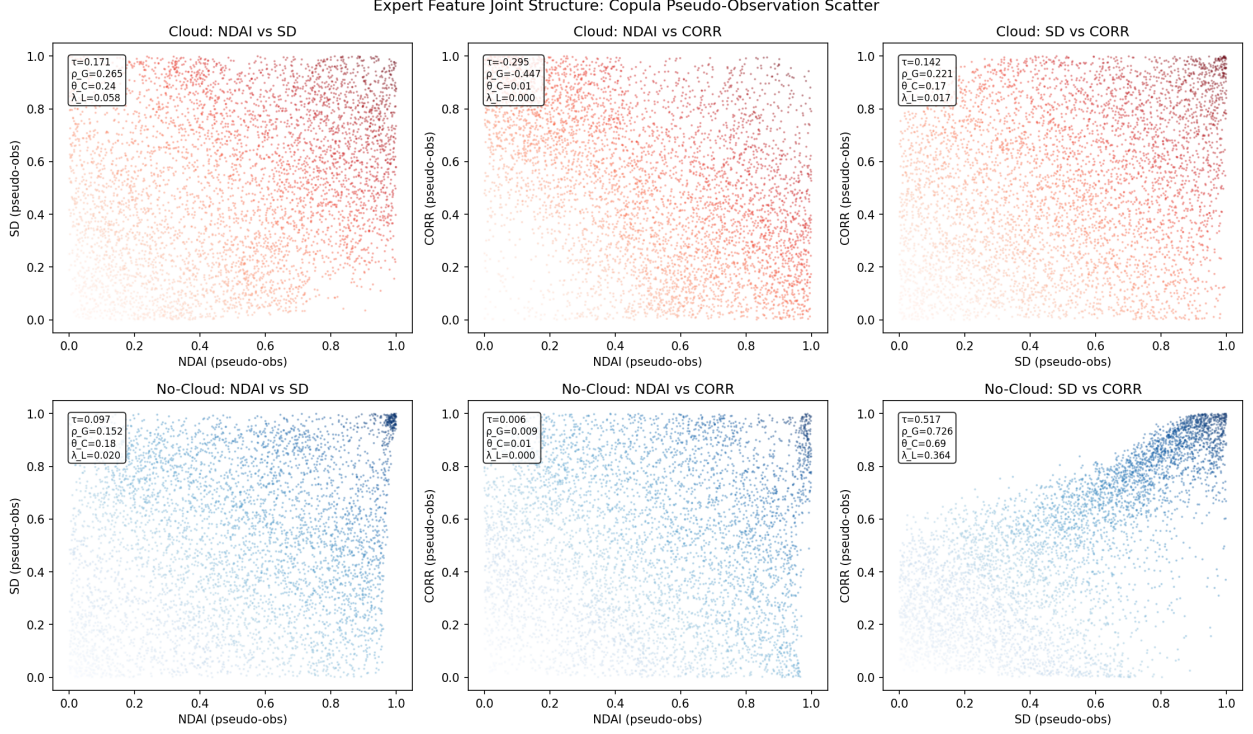


Figure 3: Feature distribution shift across the three labeled images. Top row: all labeled pixels; bottom row: cloud pixels only. CORR and Rad_AN show the most pronounced inter-image variation.

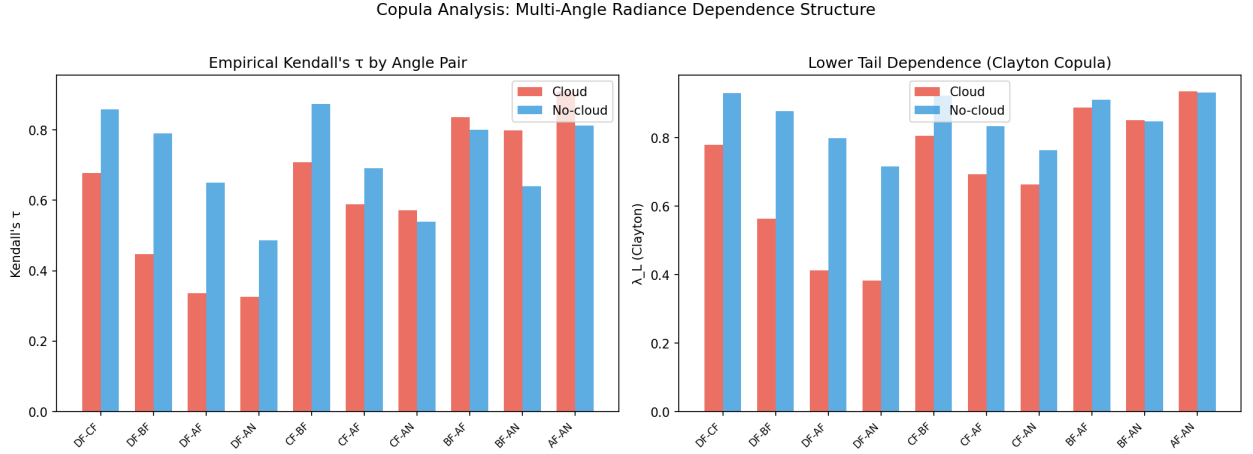
2.7 Copula Dependence Structure

To go beyond pairwise correlations, we use copula models to characterize the *joint* dependence structure of features, separately for cloud and non-cloud pixels. To our knowledge, copula-based dependence analysis has not been previously applied to MISR cloud detection; it provides a richer view of feature relationships than correlation alone, particularly in the tails of the distribution where classification decisions are most uncertain.

Figure 4 shows the results. Cloud pixels exhibit stronger upper-tail dependence in expert features (high NDAI co-occurring with high SD), while non-cloud pixels show more uniform structure. For radiance channels, non-cloud pixels have higher concordance ($\tau > 0.8$) and stronger lower-tail dependence, reflecting smooth ice/snow reflectance. Cloud pixels have weaker dependence due to parallax disruption. These class-conditional differences have direct implications for feature engineering. The upper-tail clustering in cloud pixels (high NDAI co-occurring with high SD) motivates our $\text{NDAI} \times \text{SD}$ interaction term, while the asymmetric radiance dependence motivates the angular anisotropy indices. Standard correlation analysis would miss these tail-specific patterns because Pearson correlation assumes elliptical joint distributions, which clearly do not hold here.



(a) Expert feature pairs: pseudo-observation scatter. Top = cloud, bottom = non-cloud.



(b) Radiance pairs: Kendall's τ (left) and Clayton lower-tail dependence λ_L (right).

Figure 4: Copula dependence analysis. Cloud and non-cloud pixels have distinct joint structure in both expert features and multi-angle radiances.

3 Feature Engineering

3.1 Raw Features

The 8 base features fall into two groups:

1. **Expert features** (3): NDAI measures normalized angular displacement due to cloud parallax, SD captures radiance variability across viewing angles, and CORR quantifies angular coherence. These encode cloud altitude information that is unavailable from any single angle [1].

2. **Radiance channels** (5): Rad_DF, Rad_CF, Rad_BF, Rad_AF, and Rad_AN are red-band radiance measurements from five of MISR’s nine cameras at different off-nadir angles.

3.2 Engineered Features

We expand the feature space from 8 to 71 dimensions through several transformations:

Log transforms (6 features). We apply $\log(1 + \max(x, 0))$ to SD and the 5 radiance channels. This compresses the heavy right tail of SD and radiance channels and improves linearity for logistic regression.

Angular anisotropy indices (4 features). For each off-nadir angle $j \in \{\text{DF, CF, BF, AF}\}$, we compute:

$$\text{aniso}_j = \frac{\text{Rad}_j - \text{Rad}_{\text{AN}}}{\text{Rad}_j + \text{Rad}_{\text{AN}} + \epsilon}$$

These directly encode the angular dependence of reflectance, which is the physical signature that distinguishes clouds from surface.

Adjacent-angle radiance ratios (4 features). Ratios between consecutive angular channels:

$$\text{ratio}_{j,j+1} = \frac{\text{Rad}_j}{\text{Rad}_{j+1} + \epsilon}$$

These capture local gradients in the angular radiance profile.

Pairwise interactions (3 features). Products of expert features: $\text{NDAI} \times \text{CORR}$, $\text{NDAI} \times \log(1 + \text{SD})$, and $\text{CORR} \times \log(1 + \text{SD})$. These let linear models capture nonlinear decision boundaries.

Spatial features (multi-scale neighborhood statistics). For each of the 8 raw features, we compute mean and standard deviation within 3×3 , 9×9 , and 15×15 neighborhoods, plus Sobel gradient magnitudes. These encode local texture and edge information that pixel-level features miss.

Autoencoder embeddings (32 features). A convolutional autoencoder trained on all 164 images (Section 3.3) produces 32-dimensional latent vectors that capture local spatial structure in an unsupervised way.

After winsorization at the 1st and 99th percentiles (fitted on training data only), the full feature set has 71 dimensions. Models that need it (logistic regression, KNN) include standardization in their pipeline.

3.3 Autoencoder Details

Our pipeline follows a **semi-supervised** strategy. The autoencoder trains on all 164 images (including the 161 unlabeled ones) via unsupervised reconstruction, then feeds learned representations into classifiers trained on only 3 labeled images. This lets us leverage $\sim 50\times$ more data for representation learning than a purely supervised approach.

We use a **convolutional** autoencoder rather than a standard MLP because the input is a 9×9 multi-channel image patch, and convolutions preserve the 2D spatial structure that an MLP would destroy by flattening. The encoder has three convolutional layers ($8 \rightarrow 32 \rightarrow 64 \rightarrow 64$ channels, all 3×3 kernels) with batch normalization and ReLU, reducing the spatial dimensions from 9×9 to 5×5 . The resulting $64 \times 5 \times 5 = 1600$ -dimensional feature map is then compressed through two fully connected layers ($1600 \rightarrow 128 \rightarrow 32$) to produce the embedding. The decoder mirrors this architecture using transposed convolutions. The codebase also includes an MLP baseline that flattens the $9 \times 9 \times 8 = 648$ -dimensional input and passes it through a $648 \rightarrow 128 \rightarrow 64 \rightarrow \text{embedding}$ bottleneck; we found that the convolutional variant consistently achieves lower reconstruction error because it can exploit local spatial correlations within the patch.

We conducted a two-stage hyperparameter sweep using Weights & Biases (Table 3). Stage 1 was a random search over 12 configurations spanning embedding dimensions (8, 16, 32), learning rates (10^{-4} to 10^{-3}), Gaussian noise injection ($\sigma = 0$ to 0.2), and random channel masking ($p = 0$ to 0.25). This stage identified embedding dimension 32 and learning rate 0.001 as clearly dominant: all configurations with embed=32 and lr=0.001 achieved val MSE < 0.10 , while smaller embeddings or lower learning rates stalled above 0.13.

Stage 2 was a targeted grid search over noise and masking levels within the best region, which confirmed that the simplest configuration (no noise, no masking) achieves the lowest validation MSE of 0.0909. After training, we extract 32-dimensional embeddings for every pixel by passing its local 9×9 neighborhood through the encoder.

Table 3: Two-stage autoencoder hyperparameter sweep (19 runs total, sorted by val MSE). Stage 1: random search over a broad grid. Stage 2: targeted grid search around the best region (embed=32, lr=0.001).

Embed	LR	Noise σ	Mask p	Val MSE	Train MSE	Stage
32	0.001	0.0	0.0	0.0909	0.0897	2
32	0.001	0.0	0.0	0.0915	0.0906	2
32	0.001	0.1	0.0	0.0918	0.0905	2
32	0.001	0.2	0.0	0.0920	0.0909	2
32	0.001	0.0	0.15	0.0939	0.1061	2
32	0.001	0.1	0.15	0.0945	0.1089	2
32	0.001	0.2	0.15	0.0951	0.1111	2
32	0.0005	0.05	0.1	0.0971	0.1063	1
32	0.001	0.2	0.25	0.1010	0.1257	2
16	0.0001	0.2	0.0	0.1390	0.1396	1
16	0.0001	0.0	0.1	0.1430	0.1534	1
8	0.001	0.2	0.0	0.1644	0.1623	1
8	0.0001	0.2	0.0	0.1787	0.1747	1
8	0.0001	0.2	0.25	0.1821	0.2011	1

These embeddings capture spatial patterns (cloud edges, texture gradients, surface type) that complement pixel-level features. Training on all 164 images lets the autoencoder learn from a much broader range of polar scenes than the three labeled images alone.

3.4 Feature Importance and Ablation

To understand which features matter most, we compare Random Forest Gini importance and logistic regression coefficient magnitudes. NDAI dominates in both and is by far the strongest single predictor. SD ranks second, followed by spatial neighborhood statistics (particularly SD_mean over 9×9 patches). Among raw radiance channels, Rad_AN consistently ranks highest.

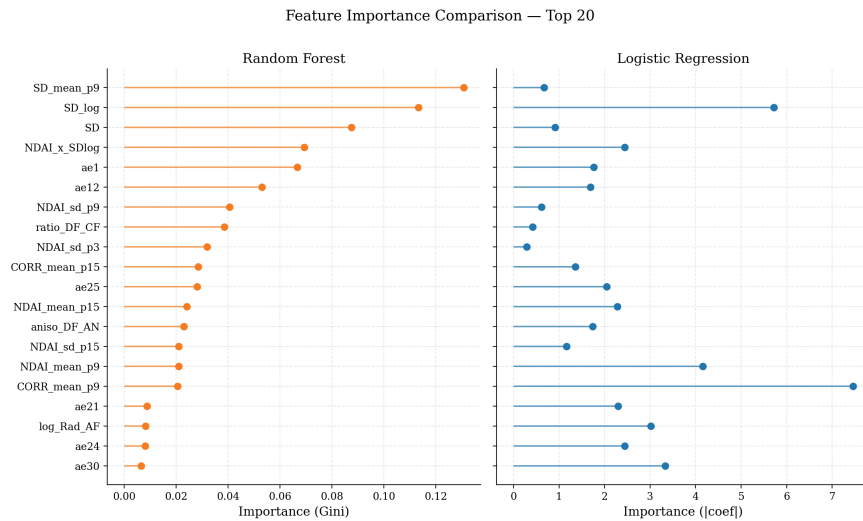


Figure 5: Top-20 features ranked by Random Forest Gini importance (left) and logistic regression absolute coefficients (right), Fold 1. NDAI dominates both rankings.

We supplement these with SHAP values for GBM (Figure 6). The key finding is that `SD_mean_p9` (the 9×9 neighborhood mean of SD) is by far the most important feature by SHAP, roughly $4 \times$ larger than any other, while Gini importance ranks NDAI first. This discrepancy arises because Gini counts split frequency, whereas SHAP measures actual prediction impact. In the beeswarm plot, high `SD_mean_p9` pushes predictions toward cloud, while high CORR spatial means push toward non-cloud.

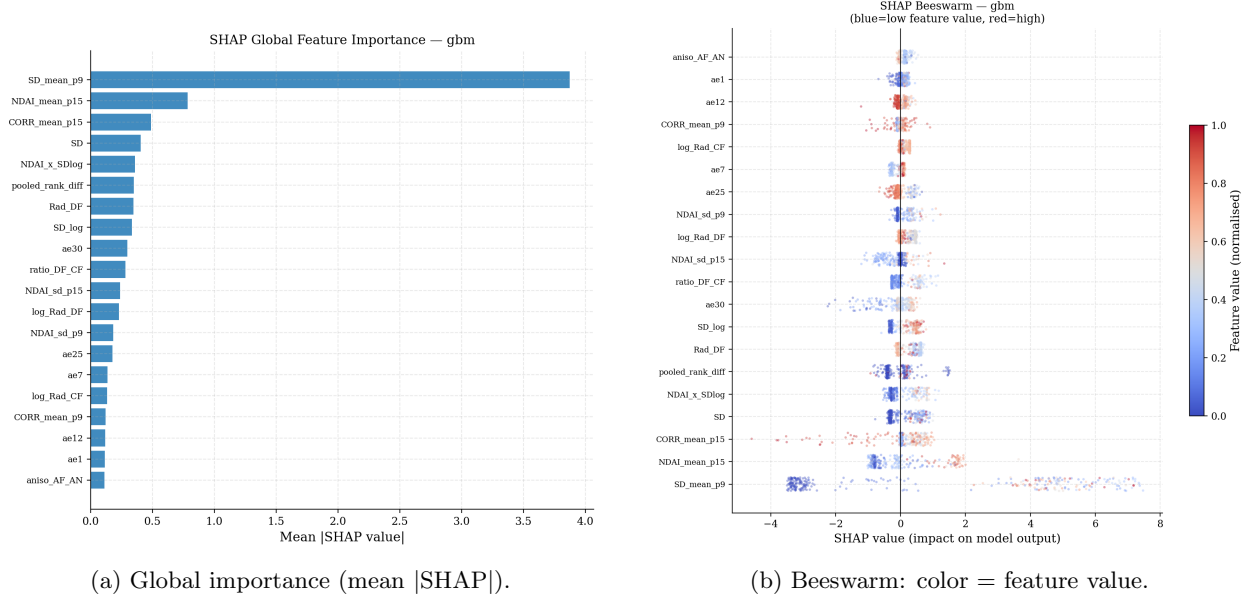


Figure 6: SHAP analysis for GBM (Fold 1). Left: `SD_mean_p9` dominates global importance. Right: high SD spatial means push toward cloud, high CORR spatial means push toward non-cloud.

Table 4 shows an ablation study where we train logistic regression on progressively richer feature sets and report 3-fold CV test metrics. Adding engineered features (transforms, interactions) improves F1 from 0.737 to 0.815 (+0.078). Autoencoder embeddings provide a further +0.051 boost (F1: 0.815 $\rightarrow$ 0.866). Notably, the 57-feature set (enhanced + AE, no spatial) achieves the *highest* F1 (0.866), while adding spatial features actually decreases F1 to 0.856, likely due to mild overfitting from the additional 14 spatial dimensions.

Table 4: Feature ablation study using logistic regression (3-fold CV test, mean $\pm$ std).

Feature Set	n	F1	PR-AUC	ROC-AUC
Expert (NDAI, SD, CORR)	3	0.743 $\pm$ 0.161	0.736 $\pm$ 0.132	0.846 $\pm$ 0.112
Radiance (5 channels)	5	0.654 $\pm$ 0.207	0.750 $\pm$ 0.160	0.773 $\pm$ 0.143
Raw (expert + radiance)	8	0.737 $\pm$ 0.152	0.737 $\pm$ 0.121	0.819 $\pm$ 0.100
Enhanced (no AE, no spatial)	39	0.815 $\pm$ 0.119	0.914 $\pm$ 0.058	0.961 $\pm$ 0.026
Enhanced + AE (no spatial)	57	0.866$\pm$0.089	0.907 $\pm$ 0.070	0.956 $\pm$ 0.032
Enhanced (full)	71	0.856 $\pm$ 0.087	0.918$\pm$0.057	0.962$\pm$0.026

4 Modeling

4.1 Classification Models

We train four classifiers using scikit-learn [5]. With only three labeled images, end-to-end deep learning risks overfitting, so we use traditional models that allow direct interpretation of feature contributions. The autoencoder (implemented in PyTorch Lightning [6]) provides deep representations without end-to-end label dependence [4]. Hyperparameters follow established defaults [3, 2]:

Logistic Regression. L2 regularization ($C = 1.0$) with balanced class weights and standardized features. The rich feature space already encodes nonlinear structure, allowing a linear model to capture complex boundaries.

Random Forest. 250 trees with `min_samples_leaf=2` and balanced subsample weighting. The number of trees follows the diminishing-returns principle: performance plateaus beyond ~ 200 estimators.

Gradient Boosted Trees (GBM). 200 trees, max depth 5, learning rate 0.1, 80% row subsampling. Depth 5 and learning rate 0.1 are standard defaults that work well in practice [3].

k -Nearest Neighbors (KNN). $k = 15$ neighbors with inverse-distance weighting. The moderate k provides smoothing against label noise while preserving local decision boundaries.

4.2 Cross-Validation Results

Table 5 presents the main results. Logistic regression achieves the highest mean F1 (0.856), followed closely by GBM (0.819). GBM leads in ROC-AUC (0.973) and PR-AUC (0.941). The standard deviations are large for all models, reflecting the difficulty of generalizing across images with different cloud patterns.

Table 5: 3-fold cross-validation results (test set). Mean $\pm$ std across folds.

Model	F1	ROC-AUC	PR-AUC	Bal. Acc.	Precision	Recall
Logistic Reg.	0.856$\pm$0.087	0.962 $\pm$ 0.026	0.918 $\pm$ 0.057	0.898 $\pm$ 0.060	0.839 $\pm$ 0.119	0.889 $\pm$ 0.101
Random Forest	0.792 $\pm$ 0.139	0.950 $\pm$ 0.051	0.932 $\pm$ 0.058	0.835 $\pm$ 0.088	0.923 $\pm$ 0.084	0.727 $\pm$ 0.203
GBM	0.819 $\pm$ 0.140	0.973$\pm$0.023	0.941$\pm$0.047	0.862 $\pm$ 0.095	0.928 $\pm$ 0.064	0.766 $\pm$ 0.215
KNN	0.782 $\pm$ 0.114	0.893 $\pm$ 0.065	0.817 $\pm$ 0.117	0.826 $\pm$ 0.085	0.849 $\pm$ 0.119	0.744 $\pm$ 0.151

4.3 Per-Fold Breakdown

Table 6 and Figure 7 break down the results by fold. The pattern is striking: all models do well on Fold 3 (test = O013490, F1 > 0.92) but struggle on Fold 2 (test = O013257). O013257 has the lowest cloud fraction (28.9%) and different cloud morphology, making it the hardest image to predict.

Table 6: Per-fold test F1 scores showing cross-image variability.

Model	Fold 1 (O012791)	Fold 2 (O013257)	Fold 3 (O013490)	Mean
Logistic Reg.	0.806	0.784	0.979	0.856
Random Forest	0.809	0.613	0.952	0.792
GBM	0.828	0.643	0.987	0.819
KNN	0.715	0.689	0.943	0.782

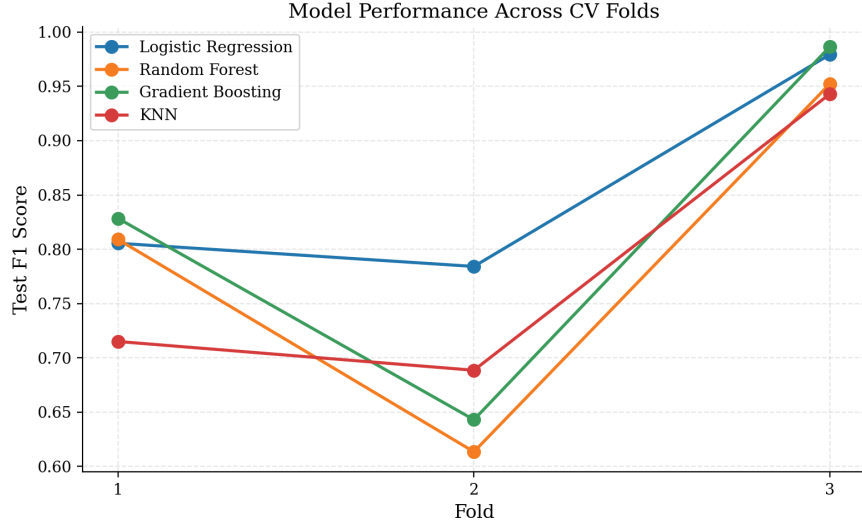


Figure 7: Per-fold test F1 for each model. All models perform best on Fold 3 (O013490) and worst on Fold 2 (O013257).

Table 7 shows per-fold precision and recall, revealing that the performance gap is driven by different failure modes. In Fold 2 (hardest), GBM has high precision (0.970) but very low recall (0.481), meaning it misses most clouds rather than hallucinating them. Logistic regression takes the opposite trade-off: lower precision (0.681) but much higher recall (0.924), catching more clouds at the cost of more false alarms. If missing clouds is worse than false alarms for a given application, logistic regression is the safer choice.

Table 7: Per-fold precision and recall, revealing different failure modes across models and folds.

Model	Precision			Recall		
	Fold 1	Fold 2	Fold 3	Fold 1	Fold 2	Fold 3
LogReg	0.868	0.681	0.969	0.751	0.924	0.990
RF	0.804	0.978	0.987	0.814	0.447	0.921
GBM	0.838	0.970	0.975	0.819	0.481	0.998
KNN	0.684	0.902	0.960	0.749	0.557	0.927

The high variance across folds reflects a real limitation, not a methodological bug. Two factors drive the poor Fold 2 performance: (1) O013257 has the lowest cloud fraction (28.9%), so the threshold tuned on higher-cloud-fraction training images overshoots, producing excessive false positives; (2) the distribution shift analysis (Figure 3) shows that O013257’s CORR and radiance distributions differ notably from the other images, meaning the angular coherence signal behaves differently in this scene. We address this with adaptive thresholding in Section 4.7.

4.4 Detailed Fold 1 Diagnostics

We show detailed diagnostics for Fold 1 (test = O012791) since it represents a moderately difficult case.

4.4.1 ROC and Precision-Recall Curves

Figure 8 shows ROC and PR curves. GBM and logistic regression dominate in area under both curves. The gap between ROC-AUC and PR-AUC for KNN is large, indicating that KNN has trouble maintaining precision at reasonable recall levels.

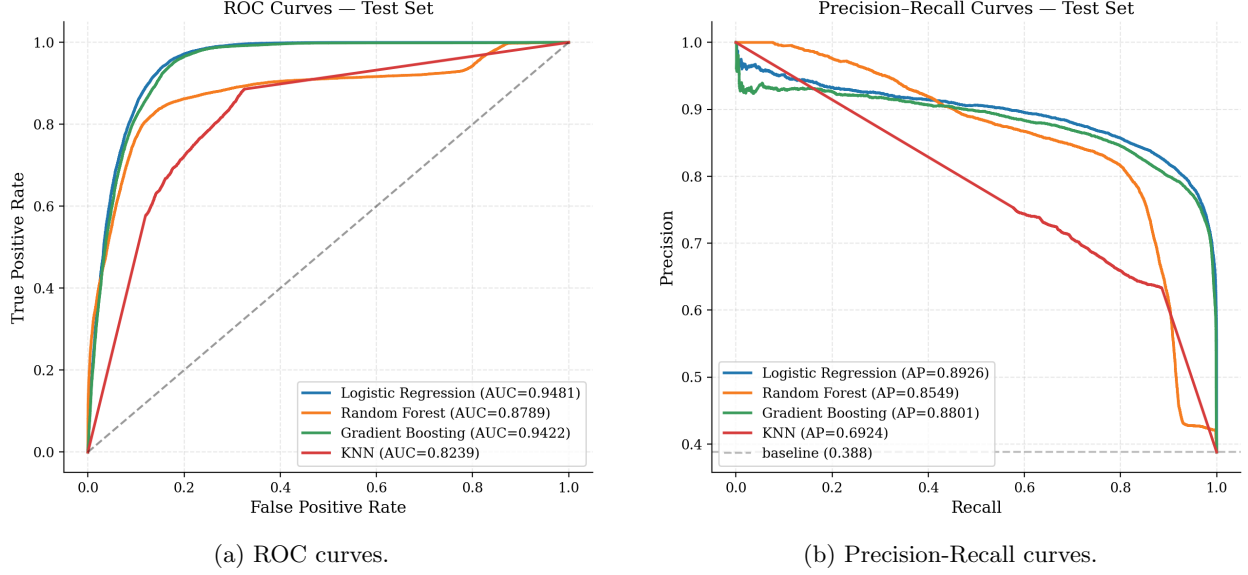


Figure 8: ROC and PR curves for Fold 1 (test: O012791). GBM and logistic regression achieve the highest AUC values.

4.4.2 Confusion Matrices

Figure 9 shows the confusion matrices for all four models on Fold 1. The different models trade off false positives and false negatives in different ways depending on their learned thresholds.

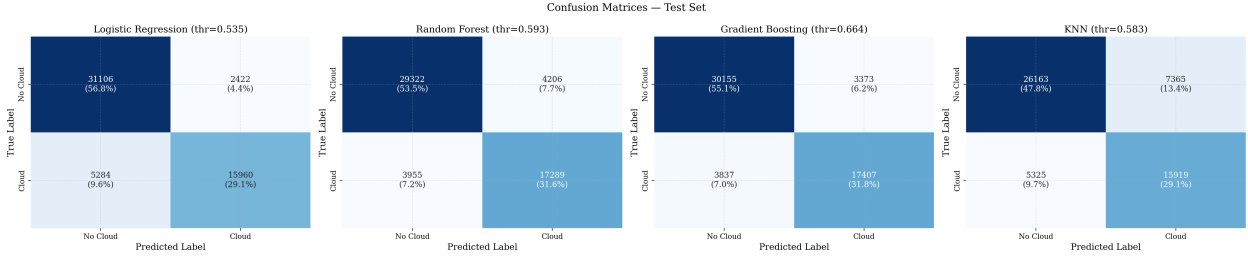


Figure 9: Confusion matrices for Fold 1 test set (O012791). Each matrix: TN (top-left), FP (top-right), FN (bottom-left), TP (bottom-right).

4.5 Best Model Selection

We select **logistic regression** as our primary model. It achieves the highest mean F1 (0.856 ± 0.087) and the lowest cross-fold variance. GBM has higher ROC-AUC (0.973 vs. 0.962), but with only 3 folds this difference is not statistically meaningful.

We use F1 instead of AUC to select the model because AUC only measures ranking quality, not the actual classification output. A model with perfect AUC can still produce poor cloud masks if its threshold is wrong, which is what happens in Fold 2.

We pick logistic regression over GBM because it is more consistent across folds (std = 0.087 vs. 0.140 for GBM), its probabilities are well-calibrated (Figure 23), and its coefficients can be read directly. When higher precision is needed, GBM (0.928 vs. 0.839) can be used instead.

4.6 Post-Hoc Analysis and Deployment

4.7 Adaptive Thresholding

The standard approach is to tune a single decision threshold on the validation set and apply it to all test images. This works when the validation and test distributions are similar, but breaks down when they differ. In our setting, the fixed threshold is tuned on training images with 29–48% cloud fraction, then applied to a test image that may have very different cloud prevalence. When the test image has less cloud than expected, the threshold is too aggressive and produces excessive false positives (e.g., RF in Fold 2 with fixed threshold = 0.628 produces $F1 = 0.613$).

We propose **adaptive thresholding**: instead of using the training-derived threshold, we apply Otsu’s method to the predicted probability distribution of each test image independently. Otsu’s method finds the threshold that maximizes the between-class variance of the probability histogram, effectively locating the natural split point between the cloud and non-cloud probability modes. This requires no labeled data at test time and adapts automatically to the image-specific cloud fraction.

Table 8 shows the results. Adaptive thresholding improves every model on average, with the largest gains on the hardest cases. For Random Forest on Fold 2, the fixed threshold of 0.628 (too high for this low-cloud image) drops to an adaptive threshold of 0.400, recovering F1 from 0.613 to 0.806, an improvement of +0.193. GBM shows a similar pattern (+0.057 on Fold 2), while logistic regression, which already has the best-calibrated probabilities, gains less (+0.005). The pattern is consistent across all four models: the worse a model’s fixed-threshold performance, the more it benefits from adaptation.

Table 8: Adaptive vs. fixed thresholding. Per-fold F1 and mean improvement (Δ).

Model	Fixed Threshold F1			Adaptive Threshold F1			Δ Mean
	Fold 1	Fold 2	Fold 3	Fold 1	Fold 2	Fold 3	
LogReg	0.806	0.784	0.979	0.818	0.790	0.977	+0.005
RF	0.809	0.613	0.952	0.805	0.806	0.960	+0.065
GBM	0.828	0.643	0.987	0.834	0.700	0.986	+0.020
KNN	0.715	0.689	0.943	0.718	0.707	0.944	+0.007

4.8 Feature Selection

The 71-feature space includes substantial redundancy: many spatial statistics are highly correlated with each other, and the 32 autoencoder dimensions likely capture overlapping information. To quantify how many features are actually needed, we rank all features by their mean absolute SHAP value from the full GBM model, then retrain GBM using only the top- k features and evaluate via the same 3-fold CV.

Table 9 shows the results. Performance increases steeply from 10 to 30 features, then plateaus. With 30 features, GBM achieves $F1 = 0.817$ and $ROC\text{-}AUC = 0.969$, nearly identical to the full 71-feature model (0.819 and 0.973). At 50 features, F1 actually exceeds the full model (0.820 vs. 0.819), suggesting that the additional 21 features introduce mild noise. The top-5 features span three distinct families: `SD_mean_p9` (spatial texture), `CORR_mean_p15` (spatial coherence), `SD` (raw expert), `NDAI_x_SDlog` (engineered interaction), and `Rad_DF` (raw radiance). Autoencoder embeddings (`ae30`, `ae25`, `ae12`) appear in positions 10–15, confirming that they contribute non-redundant information even though they rank below expert-derived features.

Table 9: Feature selection: GBM performance with top- k SHAP features.

Features k	Mean F1	Mean ROC-AUC	Notes
10	0.780	0.955	
15	0.806	0.965	
20	0.800	0.964	
30	0.817	0.969	$\approx$ full model
50	0.820	0.973	
71 (all)	0.819	0.973	baseline

The top features span all three families: spatial statistics (**SD_mean_p9**), expert interactions (**NDAI_x_SDlog**), and autoencoder embeddings (**ae30**, **ae25**), confirming that each engineering strategy contributes non-redundant information. The Lasso path (Figure 10) corroborates this: **CORR_mean_p9** and **SD_log** enter the model first as regularization decreases, while many spatial features remain near zero.

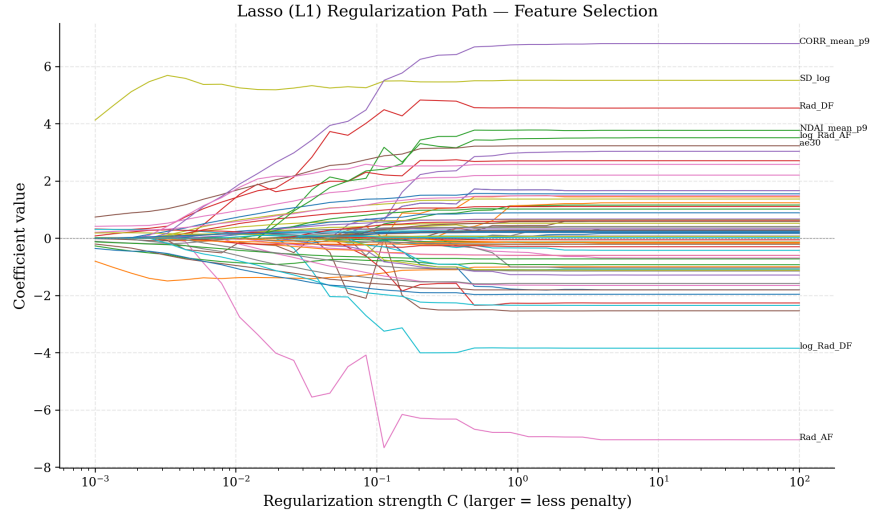


Figure 10: L1 regularization path for logistic regression. Each line traces a feature’s coefficient as penalty decreases (C increases). Core features (**CORR_mean_p9**, **SD_log**) dominate early; many spatial features remain near zero, confirming redundancy.

4.9 Spatial Error Analysis

Figure 11 shows the spatial error map for GBM on Fold 1. The key pattern is that errors are not random: they concentrate at cloud boundaries where mixed pixels produce ambiguous features. False negatives form patches of thin cloud where the parallax signal is weak; false positives occur on bright ice that mimics cloud reflectance.

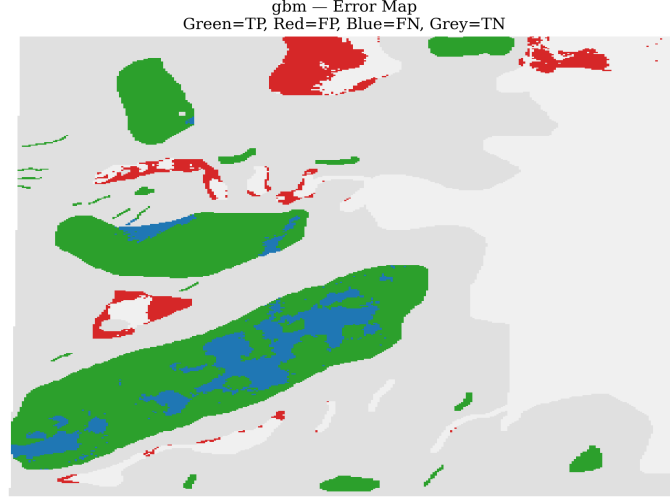


Figure 11: Spatial error map for GBM on the Fold 1 test image. Green = TP, gray = TN, red = FP, blue = FN. Errors concentrate at cloud boundaries.

4.9.1 Misclassification Feature Analysis

Figure 12 shows the feature distributions broken down by prediction outcome (TP, FP, FN, TN) for GBM. False positives (green) overlap heavily with true positives (blue) in NDAI and SD, meaning these pixels genuinely look like cloud in the feature space. False negatives (red) fall between the two class populations, corresponding to ambiguous boundary pixels where the physical signal is mixed.

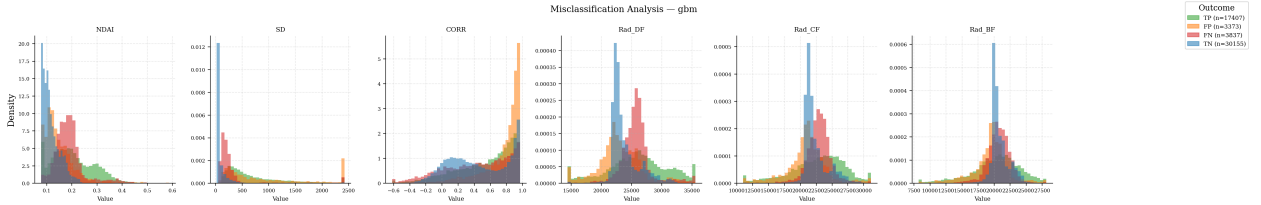


Figure 12: Feature distributions by prediction outcome (GBM, Fold 1). False positives overlap with true positives in NDAI and SD; false negatives fall between the two class populations.

4.9.2 SHAP-Based Error Diagnosis

Figure 13 shows SHAP waterfall plots for representative misclassified pixels. For the false positive, nearly all features push toward cloud because the bright, textured ice surface mimics cloud signatures. Only two autoencoder features weakly push against, suggesting the autoencoder partially recognizes the surface context.

For the false negative, high angular correlation (`CORR_mean_p15`) and autoencoder features push away from cloud, overcoming the positive contributions from SD and NDAI. This is thin semi-transparent cloud where the parallax signal is too weak for detection.

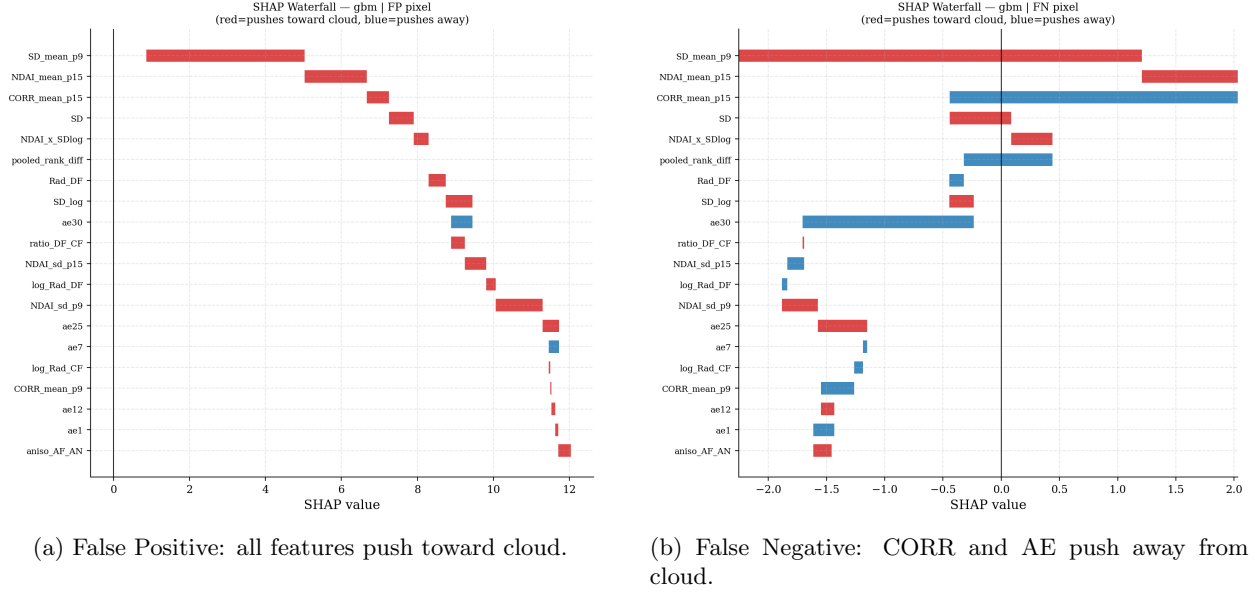


Figure 13: SHAP waterfall plots for individual misclassified pixels (GBM, Fold 1). Red = pushes toward cloud, blue = pushes away.

4.10 Probability Heatmaps

Figure 14 shows predicted cloud probabilities for all models on Fold 1. GBM produces sharper transitions while logistic regression gives smoother fields. Confidence is highest inside large cloud masses and lowest at boundaries, matching the error locations in Figure 11.

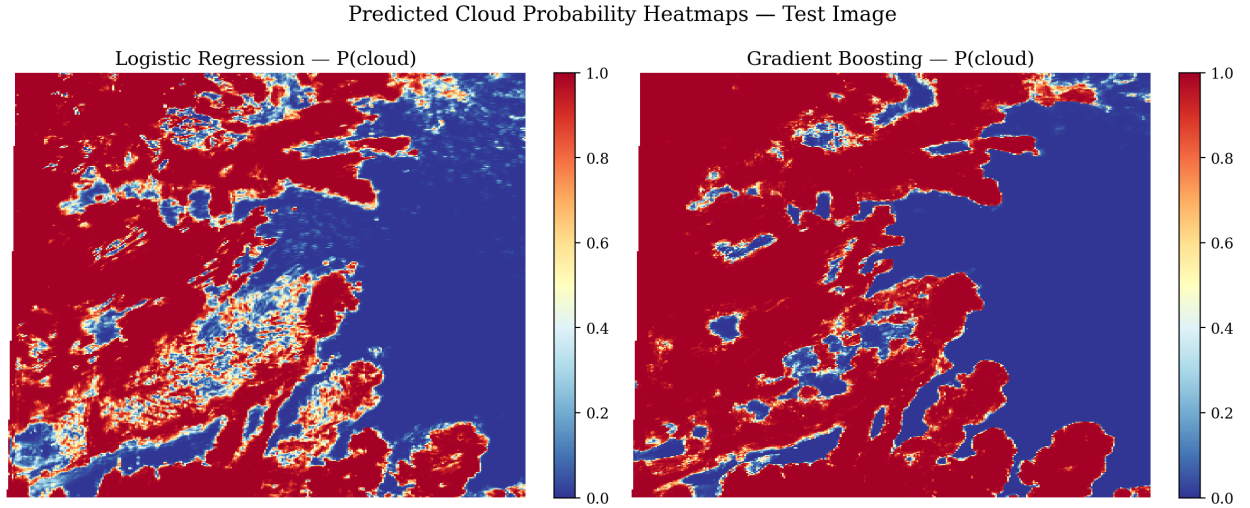


Figure 14: Predicted cloud probability heatmaps for the Fold 1 test image. Warmer colors indicate higher cloud probability.

4.11 Stability Analysis

Beyond cross-validation variance, we assess model stability using the PCS (Prediction-Computation-Stability) framework applied to logistic regression on the training data. We test four dimensions of stability:

Table 10: Stability analysis for logistic regression (PCS framework).

Test	Result	Assessment
Bootstrap (30 iters)	AUC: 0.9975 ± 0.0001 , F1: 0.9551 ± 0.0074	Stable
Preprocessing (winsorization)	AUC range: 0.993–0.998	Stable
Noise injection ($\sigma=0-0.5$)	AUC: $0.998 \rightarrow 0.980$	Stable
Feature ranking (RBO)	0.651	Unstable

Prediction stability. Figure 15 shows the bootstrap distributions: AUC concentrates around 0.9975 (std = 0.0001) and F1 around 0.955 (std = 0.007). Re-running the pipeline with a different random seed or training sample would produce nearly identical cloud masks.

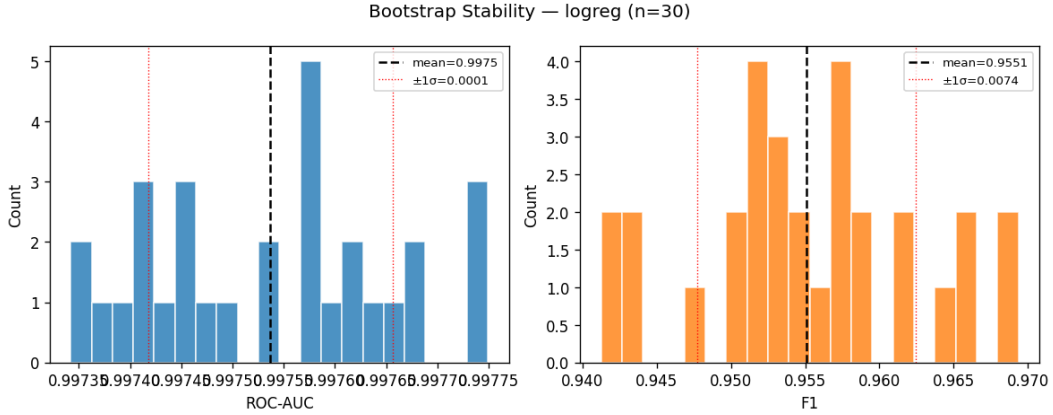


Figure 15: Bootstrap stability (30 iterations). ROC-AUC (left) and F1 (right) distributions are tightly concentrated, indicating stable predictions.

Robustness to noise. Figure 16 shows that ROC-AUC remains above 0.98 even with noise at $\sigma = 0.5$ (50% of feature standard deviation). F1 degrades more ($0.955 \rightarrow 0.901$) because the threshold becomes suboptimal under noise, but discriminative ability is preserved.

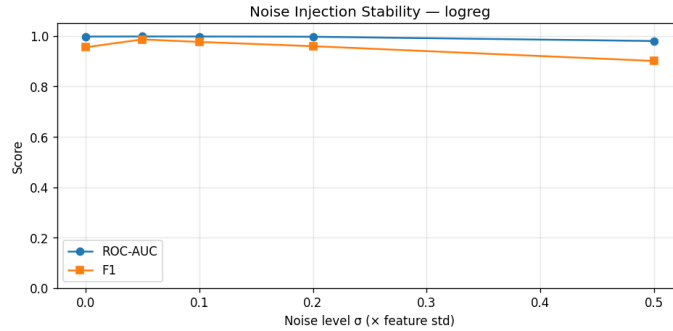


Figure 16: Noise injection stability. ROC-AUC (blue) is robust to moderate noise; F1 (orange) is more sensitive due to threshold effects.

Preprocessing robustness. Figure 17 shows performance across different winsorization levels. The 1%–99% setting achieves the best F1 (0.980), validating our default choice. Even aggressive clipping at 10%–90% only drops AUC from 0.998 to 0.993, showing that the model is not sensitive to outlier handling.

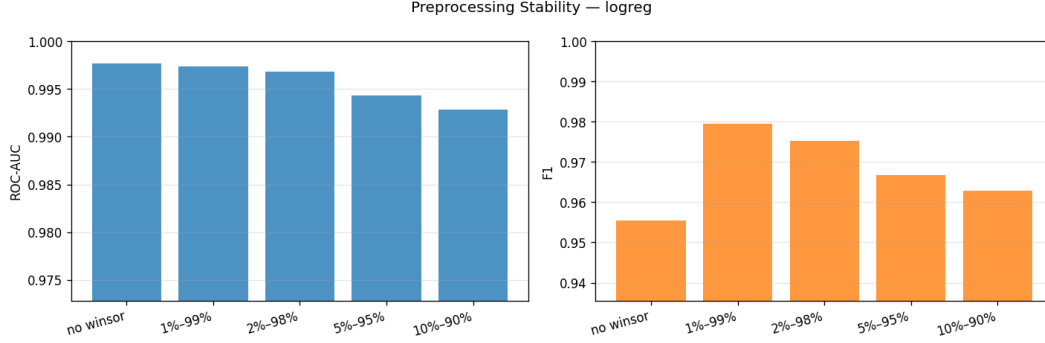


Figure 17: Preprocessing stability: ROC-AUC (left) and F1 (right) across winsorization levels. The 1%–99% default is optimal; performance degrades gracefully with more aggressive clipping.

Feature ranking instability. Despite stable predictions, feature importance rankings are unstable: RBO across 10 bootstrap runs is only 0.651. The top-10 feature set is consistent (the same features appear every time), but their ordering shuffles. This is driven by collinearity: swapping one correlated feature for another barely changes predictions but reshuffles rankings. Reducing to 30 features (Table 9) would improve ranking stability while preserving performance.

4.12 Predictions on Unlabeled Images

We deploy logistic regression to produce cloud masks for all 164 images. Figure 18 shows the distribution of predicted cloud fractions, which is bimodal with peaks at low ($<10\%$) and high ($>90\%$) coverage. The three labeled images (red bars) fall in the 30–50% range, missing both extremes. The model has never trained on near-clear or fully overcast scenes, so its predictions on those images are extrapolated.

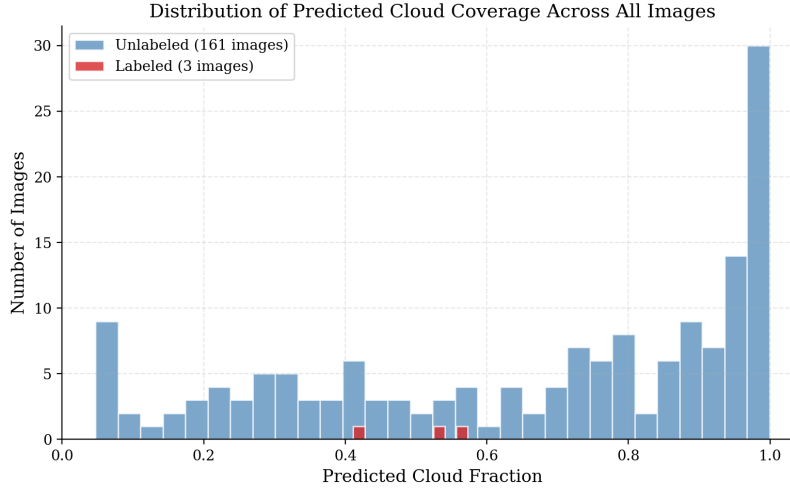


Figure 18: Distribution of predicted cloud fraction across all 164 images. Red bars mark the 3 labeled images; they fall in the mid-range, missing the bimodal extremes. Labeling images from the tails would most improve training diversity.

Figure 19 shows predicted cloud masks for a sample of unlabeled images. The predictions are spatially coherent with smooth cloud boundaries, providing a basic sanity check.

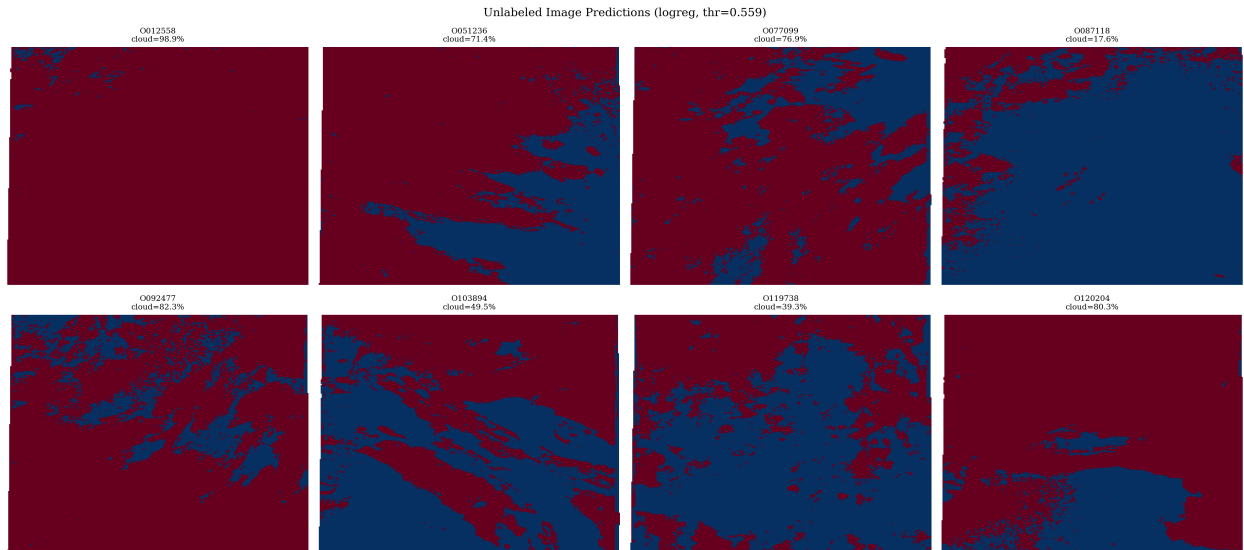


Figure 19: Predicted cloud masks for selected unlabeled images (logistic regression). Red = cloud, blue = non-cloud. Predictions show spatially coherent cloud structures.

Figure 20 shows the corresponding probability maps. Smooth gradients at cloud boundaries (rather than sharp jumps) indicate well-calibrated uncertainty. High-confidence regions correspond to thick cloud cores; boundary regions show intermediate probabilities.

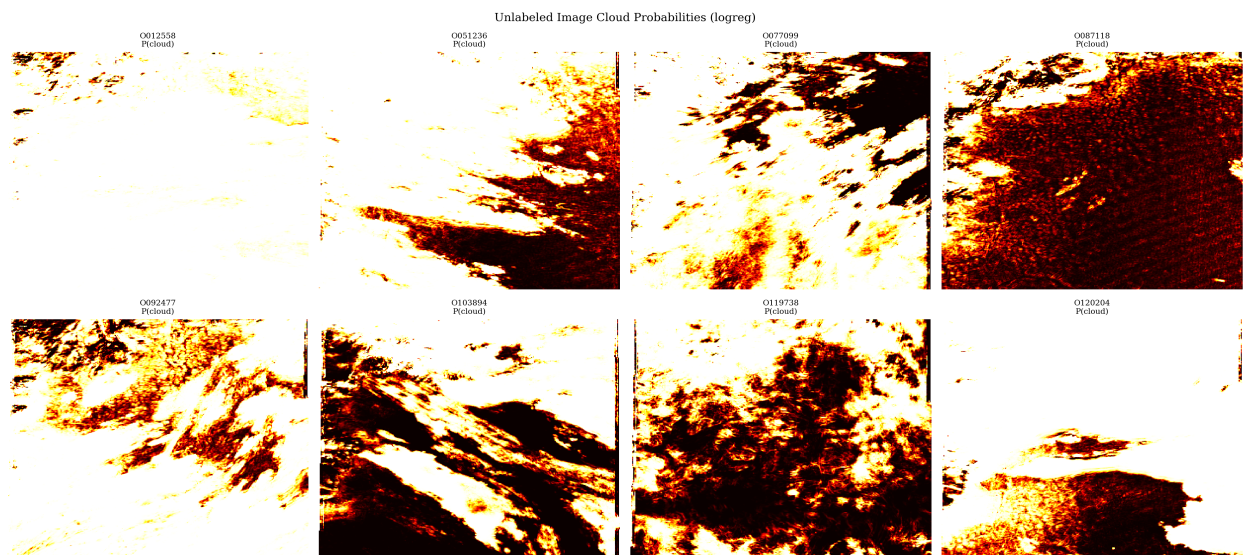


Figure 20: Predicted cloud probability maps for the same unlabeled images (logistic regression). Smooth probability gradients at cloud boundaries indicate well-calibrated uncertainty.

4.13 Discussion

As for modeling, all four classifiers achieve ROC-AUC above 0.89, and logistic regression reaches mean F1 of 0.856 with stable predictions (bootstrap AUC std < 0.001). The semi-supervised autoencoder design is a large part of this success. With only 3 labeled images, an end-to-end deep learning approach would likely overfit, but training the autoencoder on all 164 images lets us learn representations from $\sim 50\times$ more data. Its embeddings provide the single largest feature improvement in the ablation (+0.051 F1), more than any

other feature group.

As for generalization, the cross-fold variance is large (F1 from 0.64 to 0.99), and the per-fold precision/recall breakdown (Table 7) shows that different models fail in qualitatively different ways. In Fold 2, GBM has 0.970 precision but only 0.481 recall, while logistic regression has 0.681 precision but 0.924 recall. The adaptive thresholding experiment (Table 8) shows that most of this variance comes from the decision threshold, not from the model’s ability to rank pixels. Adjusting the threshold per-image with Otsu’s method recovers +0.193 F1 on Random Forest’s worst fold, without retraining at all.

As for error patterns, misclassifications cluster at cloud boundaries (Figure 11) where mixed pixels produce ambiguous features. The SHAP waterfall analysis (Figure 13) shows that false positives occur on bright ice where every engineered feature mimics cloud, while false negatives occur on thin cloud where angular correlation still looks like surface. The 71-feature space also contains substantial redundancy. Only 30 features are needed to match the full model’s F1 (Table 9), and adding spatial features beyond 57 dimensions slightly hurts performance (Table 4).

For operational deployment, we recommend using logistic regression with adaptive thresholds for balanced performance, flagging pixels with $P(\text{cloud}) \in [0.3, 0.7]$ as uncertain rather than forcing binary labels, and prioritizing expert annotation of images where the model disagrees with simple NDAI thresholding to efficiently expand the labeled training set.

5 Conclusion

We built a MISR polar cloud detection pipeline that combines multi-scale feature engineering, a semi-supervised autoencoder trained on all 164 images, and traditional classifiers evaluated under image-level cross-validation. Our main contributions are as follows.

First, the semi-supervised autoencoder design turned out to be the most impactful component. Training on all 164 images (including unlabeled ones) and extracting 32-dimensional embeddings gave the single largest F1 improvement in the ablation (+0.051), larger than any other feature group. When labels are scarce but raw imagery is abundant, unsupervised representation learning is worth the investment.

Second, the cross-fold variance (F1 from 0.64 to 0.99) initially looked like a model quality problem, but turned out to be almost entirely a threshold problem. Adaptive thresholding with Otsu’s method recovered +0.193 F1 on Random Forest’s worst fold without retraining. The models separate cloud from non-cloud well; it is the fixed decision boundary that breaks when the test image has a different cloud fraction from the training images.

Third, the copula analysis revealed that cloud and non-cloud pixels have structurally different joint dependence in the tails, not just different means. Standard correlation analysis would miss this. These tail-dependence patterns directly informed the design of the interaction features and anisotropy indices.

Fourth, 30 of 71 features are enough to match the full model’s performance. Adding spatial features beyond the 57-dimensional set slightly hurts F1 (0.866 to 0.856). Feature importance rankings are unstable across bootstrap runs (RBO = 0.651), but this is driven by collinearity among engineered features, not by model instability. Prediction stability remains high (bootstrap AUC std < 0.001).

Finally, different models fail in different ways. On the hardest fold, GBM has 0.970 precision but 0.481 recall (misses most clouds), while logistic regression has 0.681 precision but 0.924 recall (catches clouds but with more false alarms). Which model to use depends on whether missed clouds or false detections are more costly for the application.

Future work should focus on labeling more images, especially from the tails of the cloud fraction distribution (Figure 18), where the current model is extrapolating. Domain adaptation techniques and grouped feature selection could also help reduce cross-image variance and feature redundancy.

Appendix: Spatial Predictions Across Folds

Figure 21 shows spatial prediction maps for Fold 2 (test = O013257), where performance varies most across models. Figure 22 shows Fold 3 (test = O013490), where all models perform well.

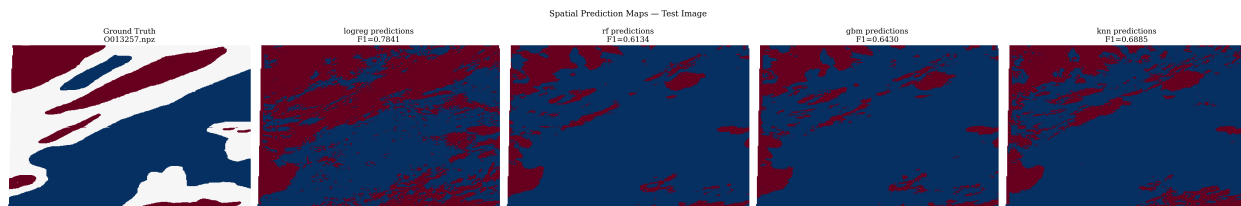


Figure 21: Ground truth and model predictions for Fold 2 test image (O013257). This is the hardest fold due to low cloud fraction and distribution shift.

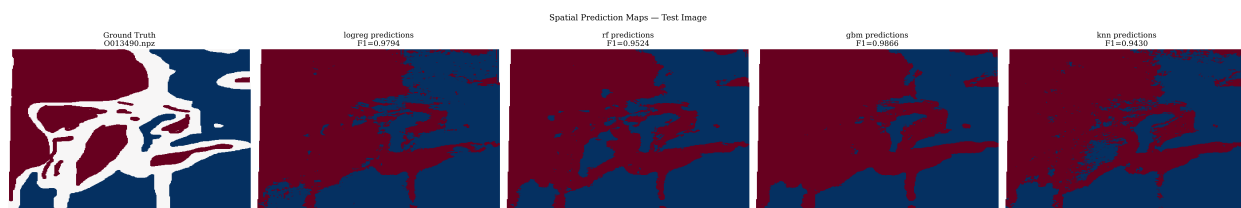


Figure 22: Ground truth and model predictions for Fold 3 test image (O013490). All models perform well on this fold.

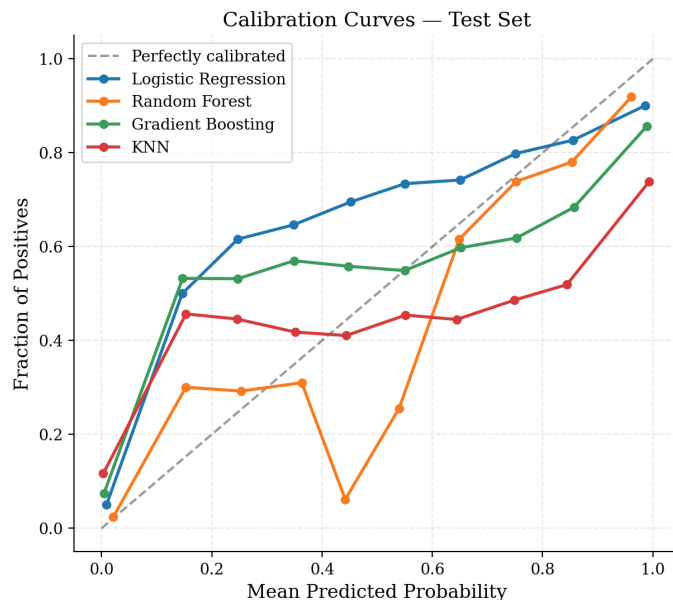


Figure 23: Calibration curves (Fold 1 test set). Logistic regression is best calibrated; tree ensembles show mild overconfidence.

A Bibliography

References

- [1] Y. Shi, B. Yu, G. Xin, and R. Bhatt, “Retrieving cloud optical depth and ice crystal size from multi-angle imaging spectroradiometer data,” 2008. (Provided as `yu2008.pdf` in the lab documents.)
- [2] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [3] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.
- [4] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [5] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [6] W. Falcon and the PyTorch Lightning team, “PyTorch Lightning,” 2019. <https://github.com/Lightning-AI/lightning>.
- [7] “MISR: Multi-angle Imaging SpectroRadiometer,” NASA Jet Propulsion Laboratory. <http://www-misr.jpl.nasa.gov/>.

B Academic Honesty

Note that this section (and the bibliography) don’t count towards the page limit.

B.1 Statement

We pledge that this report represents our own original work and analysis. All sources, including datasets, code libraries, and reference materials, are properly cited. We have not shared our code or written work with other groups, nor have we copied from any external source. We believe academic honesty is essential because the integrity of scientific research depends on transparent and truthful reporting of methods and results. Fabricated or misrepresented findings undermine the trust that the scientific community and the public place in research outcomes.

B.2 LLM Usage

Coding

LLM tools (GitHub Copilot, Claude) were used to assist with boilerplate code for data loading, visualization functions, and SLURM job scripts. All code was reviewed, edited, and tested by the team members to ensure correctness and alignment with our analysis goals.

Writing

LLM assistance was used for grammar correction and to help structure section outlines. The substantive content, analysis, interpretations, and scientific arguments in this report are our own.